

✓ 🚗 Car Price Prediction — Linear, Ridge & Lasso Regression

Dataset: Used Car Sales Data

Goal: Predict the price of a used car based on its features

Models Used: Linear Regression → Ridge → Lasso

💡 What will you learn?

- How to clean a real-world messy dataset
- Why we use log transformation on skewed data
- How Linear, Ridge, and Lasso Regression work
- How to compare models and pick the best one

✓ Step 1 — Import Libraries

Why?

Before doing anything, we load the tools we need.

- `pandas` → to load and work with data (like Excel in Python)
- `numpy` → for math operations
- `matplotlib` & `seaborn` → to make graphs
- `sklearn` → machine learning models

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error, r2_score
```

✓ Step 2 — Load the Dataset

Why?

We load our CSV file into a DataFrame — think of it as opening an Excel sheet in Python.

After loading, we always check:

- How many rows and columns are there?
- What does the data look like?

```
# Install dependencies as needed:
# pip install kagglehub[pandas-datasets]
import kagglehub
from kagglehub import KaggleDatasetAdapter

# Set the path to the file you'd like to load
file_path = "1.04. Real-life example.csv"

# Load the latest version
df = kagglehub.load_dataset(
    KaggleDatasetAdapter.PANDAS,
    "smritisinh1997/car-sales.csv",
    file_path,
    # Provide any additional arguments like
    # sql_query or pandas_kwargs. See the
    # documentation for more information:
    # https://github.com/Kaggle/kagglehub/blob/main/README.md#kaggledatasetadapterpandas
)

print("First 5 records:", df.head())
```

```
/tmp/ipykernel_5877/2219627981.py:10: DeprecationWarning: Use dataset_load() instead of load_dataset(). load_c
df = kagglehub.load_dataset(
Using Colab cache for faster access to the 'car-sales.csv' dataset.
First 5 records:      Brand      Price      Body  Mileage  EngineV Engine Type \
```

0	BMW	4200.0	sedan	277	2.0	Petrol
1	Mercedes-Benz	7900.0	van	427	2.9	Diesel
2	Mercedes-Benz	13300.0	sedan	358	5.0	Gas
3	Audi	23000.0	crossover	240	4.2	Petrol
4	Toyota	18300.0	crossover	120	2.0	Petrol

	Registration	Year	Model
0	yes	1991	320
1	yes	1999	Sprinter 212
2	yes	2003	S 500
3	yes	2007	Q7
4	yes	2011	Rav 4

Step 3 — Explore the Data (EDA)

Why?

Before building any model, we need to *understand* our data.

We check:

- Data types of each column
- Basic statistics (min, max, average)
- Missing values — are there any empty cells?

⚠ Real-world data is almost never clean. Always explore first!

df.dtypes

```

      0
Brand    object
Price   float64
Body     object
Mileage  int64
EngineV  float64
Engine Type  object
Registration  object
Year        int64
Model       object

```

dtype: object

df.info()

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4345 entries, 0 to 4344
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Brand           4345 non-null  object
1   Price           4173 non-null  float64
2   Body            4345 non-null  object
3   Mileage         4345 non-null  int64
4   EngineV         4195 non-null  float64
5   Engine Type     4345 non-null  object
6   Registration     4345 non-null  object
7   Year            4345 non-null  int64
8   Model           4345 non-null  object
dtypes: float64(2), int64(2), object(5)
memory usage: 305.6+ KB

```

df.describe()

	Price	Mileage	EngineV	Year
count	4173.000000	4345.000000	4195.000000	4345.000000
mean	19418.746935	161.237284	2.790734	2006.550058
std	25584.242620	105.705797	5.066437	6.719097
min	600.000000	0.000000	0.600000	1969.000000
25%	6999.000000	86.000000	1.800000	2003.000000
50%	11500.000000	155.000000	2.200000	2008.000000
75%	21700.000000	230.000000	3.000000	2012.000000
max	300000.000000	980.000000	99.990000	2016.000000

```
df.isnull().sum()
```

	0
Brand	0
Price	172
Body	0
Mileage	0
EngineV	150
Engine Type	0
Registration	0
Year	0
Model	0

dtype: int64

✓ Step 4 — Handle Missing Values

Why?

Machine learning models cannot work with empty (NaN) values.

We have missing values in `Price` and `EngineV`.

Solution: Drop rows where Price or EngineV is missing.

💡 `dropna()` removes rows that have missing values in the specified columns.

```
df.dropna(subset=['Price', 'EngineV'], inplace=True)
```

```
df.isnull().sum()
```

	0
Brand	0
Price	0
Body	0
Mileage	0
EngineV	0
Engine Type	0
Registration	0
Year	0
Model	0

dtype: int64

✓ Step 5 — Remove Outliers from EngineV